

LambdaScript : A Functional Programming Language

Alex Kozik
Cornell University
Ithaca, NY
ajk333@cornell.edu

Abstract—We present LambdaScript, a statically-typed functional programming language inspired by Haskell and OCaml. LambdaScript features Hindley-Milner type inference, polymorphic algebraic data types, comprehensive pattern matching, and first-class functions with closures. The language supports advanced features including mutually recursive type definitions, list comprehensions, and operators as first-class values. This paper describes both the formal semantics and the complete implementation of LambdaScript, including the design of the type system, operational semantics, and the architecture of a full interpreter pipeline consisting of lexical analysis, parser combinators, constraint-based type inference, and environment-based evaluation. The implementation is validated through 1200 unit tests covering complex real-world algorithms including red-black trees, interpreters, and graph traversal.

Index Terms—functional programming, type inference, algebraic data types, pattern matching, programming languages, formal semantics

I. INTRODUCTION

Statically-typed functional programming languages such as Haskell, OCaml, and Standard ML provide powerful abstractions for writing correct programs through algebraic data types, pattern matching, and polymorphic type inference. The Hindley-Milner type system enables programmers to write code without explicit type annotations while maintaining complete type safety through automatic type inference. However, understanding how these features interact—both formally through typing rules and operationally through implementation—requires navigating complex, mature language ecosystems with decades of accumulated features and optimizations.

This paper presents **LambdaScript**, a functional programming language designed to demonstrate that rigorous formal semantics and practical implementation can coexist harmoniously. LambdaScript features a complete Hindley-Milner type inference system, polymorphic algebraic data types, comprehensive pattern matching, and advanced features including mutually recursive type definitions. Rather than merely describing theoretical properties, we provide both a complete formal semantics and a fully implemented interpreter validated through extensive testing.

A key contribution of LambdaScript is its treatment of **mutually recursive algebraic data types**, which enable natural representation of complex data structures such as trees with forests, abstract syntax trees with multiple node types,

and graph-based structures. While languages like OCaml and Haskell support such types, their formal semantics and implementation present subtle challenges in constraint generation, type environment management, and ensuring that recursive references are properly resolved during type checking. LambdaScript addresses these challenges with a clean syntax using the `and` keyword and a carefully designed type checking algorithm that extends the standard Hindley-Milner approach.

The implementation consists of a complete interpreter pipeline: a lexer that tokenizes source code, a parser built using parser combinators for compositional grammar definitions, a constraint-based type checker implementing Hindley-Milner inference with extensions for recursive types, and an environment-based evaluator that handles closures, recursion, and pattern matching. The entire system is written in OCaml and validated through 1200 unit tests covering type inference edge cases, evaluation correctness, and real-world algorithms including red-black tree operations.

The main contributions of this paper are:

- A formal semantics for a functional language with mutually recursive algebraic data types, including complete typing rules and operational semantics
- A complete implementation architecture demonstrating how theory translates to practice, including lexer, parser combinators, constraint-based type inference, and environment-based evaluation
- Extensions to the standard Hindley-Milner algorithm to support mutually recursive types and proper handling of fixed-point types
- Validation through 1200 unit tests demonstrating correctness on complex algorithms including red-black trees, showcasing the language’s expressiveness

The remainder of this paper is organized as follows. Section II presents examples demonstrating LambdaScript’s features, building intuition before formal treatment. Section III describes the type system design, including polymorphism, algebraic data types, and type inference. Section IV provides detailed formal semantics with typing rules and operational semantics. Section V details the implementation architecture of each pipeline component. Section VI presents testing methodology and evaluation results. Section VII concludes with reflections and future directions.